SingleStore 10.4: performance & SQL productivity

August 2026 release · measured on the real-time digital marketing dataset

UPDATE / DELETE … RETURNING

Change rows and get the changed rows back in one statement. **Use it for** budget counters, queue claims, expiring rows you must report downstream.

```
UPDATE offers SET spend_cents = spend_cents + 17
WHERE offer_id = 42
RETURNING offer_id, spend_cents, budget_cents;

DELETE FROM subscriber_segments
WHERE expires_at <= NOW(6)
RETURNING subscriber_id, segment;
```

MEASURED

- One round trip instead of two: **37 → 20 ms**.
- 8 concurrent writers × 50 charges: UPDATE-then-SELECT saw only **86–153** distinct balances out of 400; RETURNING saw **400 / 400** every time.

GOTCHAS

- Columns, `*` and aliases only. An expression gives 1064: compute it in your app.
- No joins, subqueries or CTEs (1235), no `ORDER BY … LIMIT`, no `INSERT … RETURNING`.
- In a stored procedure it compiles, then fails at `CALL`.
- Every row comes back to the client: 220k deleted rows took 0.3–0.8 s vs about 0.1 s without RETURNING.

Projections

A second copy of chosen columns with its own shard and sort key, kept in sync by the engine. **Use it for** a read path that filters or groups on a key the table's layout ignores.

```
CREATE PROJECTION IF NOT EXISTS notifications_by_offer
  (SHARD KEY (offer_id), SORT KEY (offer_id, ts))
AS SELECT ts, offer_id, subscriber_id, cost_cents
   FROM notifications;

SELECT ... FROM notifications
  WITH (use_projection='notifications_by_offer')
WHERE offer_id = 42 ...;
```

MEASURED

- Rows scanned for one offer: **1.21M → 33k** (last 7 days), **3.0M → 37k** (latest 20). One partition instead of all of them.
- Cost on writes: a 300k-row bulk insert ran **2–10×** slower; a single-row insert went from 20 to 24 ms.

GOTCHAS

- For about 2 s after CREATE, the hint fails with 1877 "does not exist". Poll before use.
- Picked automatically for `GROUP BY` on its shard key. Filters need the hint. A join is only rewritten when that beats broadcasting the other side.
- Run `USE db` first: `ON db.table` is not accepted. Projections are in Preview.

DROP PROJECTION IF EXISTS

A table with a projection cannot be dropped (error 1706). Setup scripts need to drop the projection first, and now that step is safe to re-run.

```
DROP PROJECTION IF EXISTS notifications_by_offer
  ON notifications;
DROP TABLE IF EXISTS notifications;
```

GOTCHAS

- `ON <table>` is required.
- Covers a missing projection, not a missing table (1146). Check `information_schema.tables` first.
- `DROP DATABASE` is not blocked by projections.
- `IF EXISTS` is still a syntax error on `DROP INDEX` and `ALTER TABLE … DROP COLUMN`.

CPU resource pools

Named limits on CPU, memory, concurrency and run time. Now available on workspaces of every size. **Use them to** fence reports and AI agents away from latency-critical queries.

```
CREATE RESOURCE POOL campaign_reports
  WITH SOFT_CPU_LIMIT_PERCENTAGE = 20;
GRANT USAGE ON RESOURCE POOL campaign_reports
  TO CURRENT_USER;
SET resource_pool = campaign_reports;

CREATE USER ai_agent IDENTIFIED BY '<14+ chars>'
  WITH DEFAULT RESOURCE POOL = ai_agent;
```

MEASURED (AD-SERVING P95, 4 REPORTS RUNNING, 6 RUNS)

- Reports in `default_pool`: **100–156 ms**. In their own pool: **42–45 ms**. Serving alone: 23–24 ms. Reports completed just as many runs.

GOTCHAS

- The creator gets no USAGE: `SET` fails with 1231 until you `GRANT`. Switching back to `default_pool` needs USAGE on it too.
- `DROP RESOURCE POOL` fails while any grant exists: `REVOKE` first.
- HARD limits need `SET GLOBAL resource_governor_cpu_limit_mode = 'HARD'`, and every pool's CPU % must then total ≤ 100.
- A percentage limit does not slow a query running alone. `MAX_CONCURRENCY` counts per aggregator. `QUERY_TIMEOUT` is coarse (5 s killed at 7–11 s, error 2293).
- Not available on Free Starter (1227).

Figures are from the smallest Standard workspace (S-00) on engine 10.4.0: 3M ad requests, 500k purchases. Use them for direction, not as benchmarks. Try it yourself in the notebook **"Tune a Real-Time Ad Platform with Resource Pools, RETURNING and Projections"** in SingleStore Spaces.